The Cobbler Project

How to edit the code: where everything lives and how to change it

The app is plain HTML, CSS and JavaScript: no frameworks, no build step and no server code. Edit a file in any text editor, save it, and reload the page in your browser to see the change. Every file starts with a comment explaining what it does, and the code is commented throughout.

1. Running the app
2. Folders and files
3. How a page is put together
4. Where data is saved
5. Styles and themes
6. Adding a new page
7. Shared building blocks
8. Common changes
9. Exports and file formats
10. The guides and the app ZIP
11. Tips

1. Running the app

Double-click `pages/index.html`, or (recommended) run a small local web server so that file storage, the default resources table and the app ZIP all work. In Terminal:

```
cd path/to/the-cobbler-project
python3 -m http.server 8123
```

Then open `http://localhost:8123/pages/index.html`. Stop the server with Ctrl+C.
(`.claude/launch.json` holds the same command for Claude Code's preview.)

> After editing, if the browser shows an old version, do a hard reload: Cmd+Shift+R (Mac) or Ctrl+Shift+R (Windows).

Putting it online with GitHub Pages

The app is plain files, so GitHub Pages can host it for free. Only the code goes online: projects stay in each person's browser and sync folder. `.gitignore` keeps backups, shared files and calendar files out of the repository; `index.html` at the top sends visitors to `pages/index.html`; `.nojekyll` tells GitHub to serve the files as they are.

1. Make a free account at github.com, then a new repository (e.g. `the-cobbler-project`), Public (free Pages needs a public repository, or a paid plan for a private one). Don't add a README.
2. In Terminal, in the project folder:

```
git init -b main
git add .
git commit -m "First version"
git remote add origin https://github.com/YOUR-NAME/the-cobbler-project.git
git push -u origin main
```

Git asks you to sign in: use GitHub Desktop or the GitHub CLI, or a personal access token as the password.

3. On GitHub: the repository › Settings › Pages › Build and deployment › Source: *Deploy from a branch*, Branch: `main`, folder `/ (root)` › Save.
4. A minute later it's at `https://YOUR-NAME.github.io/the-cobbler-project/`.
5. To publish changes: `git add .`, `git commit -m "What changed"`, `git push`.

> Everything in the repository is public, including the code and the change log. Projects are never in it. Every page under one github.io address shares browser storage, so keep other apps you make on different repositories in mind (this app saves under `cobblerProjects2`).

2. Folders and files

Where	What
`pages/`	One HTML file per page (the sidebar links to these).
`resources/css/style.css`	All the styles, in numbered sections (listed at the top of the file).
`resources/js/`	The scripts: one per page, plus shared ones (below).
`default-resources-cobbler-project.csv`	The Resources page's default table (Name, Description, Link). At the top so it's easy to replace on GitHub.
`resources/data/`	The generated copy of the default resources, for opening pages by double-clicking.
`resources/assets/`	The logo, and the app icons for installing (`icon-192.png`, `icon-512.png`).
`resources/docs/`	These guides (HTML) and their PDFs.
`resources/vendor/abcjs/`	abcjs (MIT licence), which draws sheet music on the Music page.
`index.html`, `.gitignore`, `.nojekyll`	For hosting on GitHub Pages (section 1).
`manifest.webmanifest`, `sw.js`	Installing and offline use: the web app manifest (name, icons, colours) and the service worker that keeps a copy of the app (section 4).
`tests/run.mjs`, `package.json`, `eslint.config.mjs`, `.github/workflows/tests.yml`	The checks that run on every push (section 4, *Self-tests*).

`update-resources.sh`	Regenerates the default resources copy (for double-clicked pages) after editing the CSV locally.
`make-docs.sh`	Regenerates the guide PDFs after editing the guides.
`change-log.md`	What changed, and when.

Scripts on every page

File	What it does
`lz-string`	Compression (`resources/vendor/lz-string/`, MIT licence). Loads before projects-core.js, which also fetches it itself if a page lacks the tag.
`util.js`	Small helpers every script shares as `window.Cobbler`: `el`, `$`, `newId`, `slug`, `plural`, `pad`, `todayISO`, `download`, `csvField`, `statusLine`. Loads first.
`projects-core.js`	Stores every project and offers the Projects API (section 4).
`boot.js`	Loads the page's remaining scripts (listed on its tag) once the projects have been read from the database, then fires `DOMContentLoaded` again for them and `cobbler-ready`.
`header.js`	Builds the sidebar, the ☰ button and "Return to top"; registers the service worker. The sidebar's links are listed here.
`project-ui.js`	The Projects button and panel, the + button and the New Project dialog.
`color-themes.js`	Applies the theme (light, dark, rainbow, system) and wires the Settings buttons.
`comments.js`	The comments drawer, and the comment threads other pages reuse.
`file-store.js`	Uploaded files in IndexedDB.
`share.js`	Sharing: the share file, email, changes since last share, undoing a merge, the Claim button.
`sync.js`	The sync folder, the sync note in the sidebar, share reminders, the differences sync settled, the name prompt, the read-only notice and the storage warning.
`account.js`	The person button: sign in with your name, change it, sign out (`cobblerMe` and `commentAuthor` in localStorage); asks once on a first visit.
`undo.js`	Cmd/Ctrl+Z and Cmd/Ctrl+Shift+Z on every page (the steps themselves are kept by projects-core.js).
`ideas-panel.js`	The lightbulb button and the ideas panel on the right (the Ideas page's cards, on any page; `ideasPanelOpen` in localStorage remembers whether it's open).

Page scripts

Page	Scripts
Project Tracker	`project-tracker.js`
Projects	`projects.js`, `report.js` (project details document), `projects-sync.js` (Sync folder and Activity sections)
Home	`dashboard.js` (uses `calendar.js`'s `CobblerCalendar.datesFor`)
Self-tests	`tests.html`, `tests.js`

Writing	`writing-core.js` (state, tabs, templates), `writing-draft.js`, `writing-check.js`, `writing-story.js`, `writing-reporting.js`, `writing-revisions.js`, `writing-business.js` (Business and Goals)
Screenplay	`screenplay.js` (editor, page layout, PDF, Final Draft and Fountain)
Music	`music-core.js`, `music-songs.js` (the song sketchbook), `music-business.js` (gigs, releases, samples); abcjs for notation
Team · Ideas · Pitches · Packaging · Research · Calendar	`team.js` and `team-tasks.js` (tasks, credits) · `ideas.js` · `pitches.js` · `packaging.js` · `research.js` · `calendar.js`
Interviews	`interviews.js`
Media & Transcripts	`media-core.js` (shared engine), `media.js`, `text-edit.js`, `timeline-export.js`
A/V Script	`av-script.js`
Storyboarder	`storyboarder.js`
Shot List	`shot-list.js` (shot numbers from scene and order; shots from storyboard frames and screenplay scene headings; CSV and PDF)
Zine Planner	`zine-planner.js`, `zine-sketch.js`, `zine-critique.js`
Data Analysis	`data-analysis.js`
Resources · Settings	`resources.js` · `settings.js`

3. How a page is put together

Every page has the same shape. Copy an existing page to start a new one:

```
<head>
  <link rel="stylesheet" href="../resources/css/style.css">
  <link rel="manifest" href="../manifest.webmanifest">
  <meta name="theme-color" content="#FAD39D">
</head>
<body>
<script src="../resources/vendor/lz-string/lz-string-1.5.0.min.js"></script>
<script src="../resources/js/util.js"></script>
<script src="../resources/js/projects-core.js"></script>
<script src="../resources/js/header.js"></script>
<script src="../resources/js/color-themes.js"></script>
<script src="../resources/js/boot.js"
        data-scripts="project-ui.js comments.js file-store.js share.js sync.js
undo.js account.js ideas-panel.js my-page.js"></script>

<main>
  <h1>My Page</h1>
  …
</main>
</body>
```

- The first tags run as the page loads: the helpers, `projects-core.js` (which starts reading the projects), the sidebar and the theme (no `defer`, so the right colours apply before the page appears).
- Everything that needs the projects is listed on `boot.js`'s tag, in order, and is loaded once they've been read. Those scripts still draw the page on `DOMContentLoaded` (boot.js fires it again for them), so a page script looks the same as it always did.
- To redraw without a reload (after an undo, or when changes arrive), a page listens for `projects-refresh` on `document`, reads its data again, re-renders and sets `event.detail.handled = true`. Pages that don't are reloaded instead.
- `<main class="is-wide">` lets a page use the full width beside the sidebar (the tracker, the editing pages).
- Scripts build elements with a small `el(tag, props, ...children)` helper; text is always added as plain text, so nothing typed by a user can inject code.

4. Where data is saved

Projects are saved in the browser's database, IndexedDB, in a database called `cobbler-projects`: one record per project in the `projects` store (key: the project's id) and the active project's id in `meta`. Only the projects that changed are written, a moment after the save (several saves in a row become one write; `Projects.flushed()` waits for them). `Projects.ready` resolves once the projects have been read; `boot.js` waits for it before loading the page's scripts. Tabs tell each other about saves over a `BroadcastChannel` of the same name, and swap the new data in (`projects-data-changed` on `document`; sync.js then asks the page to redraw).

Older saves in `localStorage` (key `cobblerProjects2`, or `cobblerProjects` before that; plain JSON, or lz-string compressed with an `LZ1:` prefix from 50,000 characters) are moved into the database on first load and then removed. Where IndexedDB isn't available (some private windows), the app keeps using localStorage the old way; `Projects.storageMode()` says which. If saved data can't be read, nothing is saved over it. Each page keeps its data inside the project, under its own key:

```
project.data.tracker        Project Tracker details and goals
project.data.team           Team
project.data.tasks          Team: tasks (comments can also have an assignee)
project.data.research       Research notes
project.data.interviewees   Interviews: people
project.data.questions      Interviews: question bank
project.data.pitches        Pitches
project.data.packaging      Packaging (titles, thumbnails, SEO)
project.data.ideas          Ideas (and ideaColumns)
project.data.calendarEvents Calendar: your events
project.data.media          Transcripts and clips, Resolve markers (markerSets),
                            where everyone keeps the media (locations)
project.data.edits          Paper edits
project.data.storyboards    Storyboards
project.data.shotLists      Shot lists (name, date, call time, location, shots: [{
scene, size, angle, move, … done }])
project.data.zines          Zines
project.data.datasets       Data Analysis datasets (details only)
project.data.resources      Your resources table
project.data.comments       Page comments
project.data.drafts         Writing: drafts (parts, scenes, claims, versions,
feedback)
project.data.storyBible     Writing: story bible
project.data.sourceLog      Writing: sources and contact logs
project.data.recordsRequests  Writing: records requests
project.data.eventTimeline  Writing: event timeline
project.data.submissions    Writing: submissions and invoices
project.data.corrections    Writing: corrections log
project.data.writingLog     Writing: words written per person per day
project.data.screenplays    Screenplay: scripts (title page details, elements: [{
id, type, text }])
project.data.songs          Music: songs (lyrics, ABC sheet music, splits,
sessions, versions)
project.data.gigs           Music: gigs
project.data.releases       Music: release plans
project.data.samples        Music: sample clearances
project.data.claims         Who's working on what
project.data._deleted       Bookkeeping: deleted items, so deletions sync
project.data._activity      Bookkeeping: the activity feed
```

The Projects API (projects-core.js)

Call	What it does

`Projects.getData(key, fallback)`	Read this page's data from the active project.
`Projects.setData(key, value)`	Save it (and mark the project as edited).
`Projects.getDataFor(id, key, fallback)` / `setDataFor`	The same, for any project.
`Projects.list()` · `Projects.info()`	Every project (names, ids, archived), or with details for cards.
`Projects.activeId()` · `activeName()`	The project in use.
`Projects.create(name, { activate })`, `renameById`, `duplicate`, `removeById`, `setArchived`, `switchTo`	Managing projects.
`Projects.collect(key)`	Every project's value for a key (used to check shared files).

Undo and redo

Every `setData` also records what it replaced (in `sessionStorage` under `cobblerUndo`, per tab, up to 60 steps; saves to the same key within 1.5 seconds are one step). `Projects.undo()` / `redo()` put a key's value back through the same tracking as any save, so an undo syncs to collaborators. Steps for anything changed by a merge or another tab are dropped. `undo.js` first dispatches `pagehide` (pages save pending changes on it), undoes, then calls `Projects.refresh()`, which sends `projects-refresh`: the page reads its data again and redraws in place. A page without a listener is reloaded instead (after `Projects.freeze()`, so it can't save its old copy). Saving the very fallback a page read (its default columns, say) isn't recorded as a step.

Self-tests

`tests.js` loads the app's scripts into a sandbox (`new Function` with a fake `localStorage`, `sessionStorage`, `document` and a controllable `Date`), so tests never touch real projects. Add a test with `test("name", async () => { ok(...); eq(...); });` `withCore()` gives a fresh projects-core and `load(file, env, extras)` loads another script. The database tests use a scratch IndexedDB of their own (`window.COBBLER_DB_NAME`) and delete it afterwards.

They also run on every push, on GitHub (`.github/workflows/tests.yml`): `npm run lint` (ESLint, `eslint.config.mjs`, which lists the globals each script gets from the others) and `npm test` (`tests/run.mjs`: serves the repository, runs the self-tests page in a headless browser and opens every page, failing on any error). To run them yourself: `npm install && npx playwright install --with-deps chromium && npm run lint && npm test`.

Offline use and installing

`sw.js` (at the root, registered by `header.js` once a page has loaded) keeps a copy of every file in its `PRECACHE` list and answers requests from that copy while fetching a fresh one for next time. Projects aren't involved: they're in the database. **After changing any file, bump `VERSION` in sw.js** and add new files to the list: the new worker makes a new cache, takes over, drops the old one, and header.js offers a reload. `manifest.webmanifest` (linked from every page) gives the app its name, icons and colours for installing.

Collaboration: change tracking, merging and sync

- **Saving** (`setData`) compares what the page saves with what it last read: changed items in lists get `_mt` (time) and `_mb` (your name, `Projects.me()`); removed items are recorded in `_deleted`; a line goes into `_activity`. If the data changed underneath the page since it read it (a collaborator's changes arrived, or another tab saved), the page's own changes are merged into what's there instead of overwriting it.
- **Merging** is `Projects.mergeData(mine, theirs, base)`: lists of things with an `id` are matched item by item, plain lists are combined, one-sided changes (compared with `base`, the last version both had) are taken, and anything changed on both sides becomes a difference (the newer `_mt` wins by default). Afterwards anything in `_deleted` is removed unless it was changed after it was deleted. New pages work with this automatically as long as list items have an `id`.
- **Sync** (`sync.js`): the chosen folder's handle is kept in IndexedDB. Each project has a folder `Name [share id]` with `people/<name>-<device>.cobbler.json` (each person writes only their own) and `files/` (uploads). Every 30 seconds, and soon after a save, it merges everyone else's newer files (using the last version merged from each person as the base) and writes yours if it changed. It needs the File System Access API (Chrome, Edge).
- **Read-only copies** (`project.readOnly`): `setData` refuses everything but `comments`. Their feedback file (`kind: "feedback"`) carries only comments, and the Projects page merges just those.
- **Differences** from `mergeData` carry a `path` (e.g. `["research", { id }, "status"]`); sync keeps the ones it settled in `localStorage cobblerConflicts:<project>`, and `Projects.setAtPath` switches one later.
- **Locked versions** are in IndexedDB under `lock:<project>:<id>` (share.js).
- The last version shared or received, the last version merged from each person, and a copy from before each merge (for *Undo last merge*) are kept in IndexedDB under `share-base:`, `sync-base:` and `undo-merge:`.

Files (critique uploads, storyboard photos, datasets) are kept by `file-store.js` in their own IndexedDB database (`cobbler-files`): `FileStore.put(id, blob)`, `get(id)`, `remove(id)`. Project data stores only the file's id. Settings › Storage › *Find unused files* deletes ids no project mentions, except the bookkeeping ones (`sync-`, `share-base:`, `undo-merge:`, `lock:`, `transcripts-folder-handle`).

Transcripts kept outside the project (`media-core.js`): a transcript can have `storage: { where: "browser" | "folder", key, folder, path, at, by }` and `storageLog: [{ at, by, where, folder, path }]`. Its words are then in FileStore under `key` (`tx-<id>-<version>`; the key changes whenever the words do, so share and sync carry the new copy like any uploaded file) and, for a folder, also in `<folder>/Cobbler transcripts/<project> [id]/<title> [id].json` (the folder handle is in FileStore as `transcripts-folder-handle`). The project keeps `segments: []` plus `paragraphs` and `length`. `MediaCore.hydrate()` reads the words into memory (every page with media-core.js does this on load and fires `transcripts-loaded`), `loadMedia()` returns a copy with them filled in, `packMedia()` takes them out before saving, and `moveTranscript(t, where, { root })` moves and logs.

Per-browser settings (not part of projects) also live in localStorage: `selected-theme`, `navOpen`, `commentAuthor`, `sketchTools`, `trackerView`, `calendarView`, `storyboardView`, `zineSketchView`, `zinePaper`, `defaultTableHidden`, `textEditCurrent:<project>`, `dataAnalysisCurrent:<project>`, `writingCurrent:<project>`, `writingGoal`, `writingCurrency`, `cobblerMe` (your name),

`cobblerDevice`, `shareReminderDays`, and the sync bookkeeping `cobblerSyncSeen` / `cobblerSyncWrote`.

> To look at saved data, open the browser's developer tools (Cmd+Option+I / F12) → Application → IndexedDB → cobbler-projects (and Local Storage for the per-browser settings). Back up projects before experimenting.

5. Styles and themes

`style.css` is split into numbered sections, listed at the top of the file (sections 1–4 are fonts, colours, base text and layout; 5 is shared components; later sections belong to pages). Find a page's section by searching for its name in capitals, e.g. `PROJECT TRACKER`.

Colours are variables

```
--color-background   page and card surfaces
--color-text         text and borders
--color-on-text      text sitting on a --color-text fill
--color-accent … --color-accent-5   peach, pink, purple, blue, grey
```

Change a value in section 2 and it changes everywhere. Each theme redefines these variables: `[data-selected-theme="light"]`, `"dark"` and `"rainbow"`. Rainbow's extra rules (which accent each surface uses) are in the Rainbow section. *System Default* is handled in `color-themes.js`: it picks light or dark from the computer's setting.

Adding a theme

1. Add a `[data-selected-theme="mytheme"] { … }` block in section 2 with the colour variables.
2. Add `"mytheme"` to `THEMES` in `color-themes.js`.
3. Add a button to `pages/settings.html`: `<button data-theme="mytheme" aria-pressed="false">My Theme</button>`.

6. Adding a new page

1. Copy a simple page (e.g. `pages/research.html`) to `pages/my-page.html`; change the `<title>`, the `<h1>` and the page script.
2. Add a link in `header.js`: `<li><a href="my-page.html">My Page</a></li>` (the list is alphabetical).
3. Write `resources/js/my-page.js`: read with `Projects.getData("myPage", [])`, save with `Projects.setData("myPage", value)`.
4. Add a numbered section at the end of `style.css` and list it in the contents at the top.
5. To include the page's data in project details exports, add a section in `report.js`; to show its dates on the Calendar, add them in `datesFor()` in `calendar.js`.

6. Note the change in `change-log.md`.

7. Shared building blocks

File	Use it for
`sketch-core.js`	Drawing and text on a page: `Sketch.createEditor({ save, onRestore })`, `editor.page(page, ratio)`, `Sketch.preview(page, ratio)`. Used by zines and storyboards.
`media-core.js`	Timecode, transcript and timeline readers, paper-edit blocks and starter structures.
`comments.js`	`CobblerComments.mount(container, { getThreads, save })` shows comment threads anywhere.
`doc-export.js` + `zip.js`	Documents as PDF (print) or Word (.docx) from a simple list of blocks; ZIP files.
`file-store.js`	Saving uploaded files.
`Projects.shareIdOf(id)` · `replaceData(id, data)`	Sharing: every copy of a shared project keeps the same share id; merging replaces a project's data.

Sharing and merging (share.js)

`ProjectShare.download(id)` writes a project with its uploaded files (as data URLs) to a .json file. `ProjectShare.merge(mine, theirs, base)` combines two copies: lists of things with an `id` are matched item by item, plain lists are combined, and anything changed differently on both sides becomes a difference to choose. `base` is the last version sent or received, kept in file storage under `share-base:<share id>`: with it, one-sided changes and deletions are applied without asking. New page data works with merging automatically as long as list items have an `id`.

Print layouts: anything inside an element with class `print-sheet` prints on its own while it has content (style.css section 27).

8. Common changes

To change…	Edit
Sidebar links	`header.js` (headerTemplate)
Tracker statuses and stages	`STATUSES` and `STAGES` at the top of `project-tracker.js`
Interview stages, starter checklist, starter questions	`STAGES`, `CHECKLIST`, `STARTER_QUESTIONS` in `interviews.js`
Pitch statuses and formats	`STATUSES`, `FORMATS` in `pitches.js`
Writing templates (beat sheets, article structures)	`TEMPLATES` in `writing-core.js`

Style rules, clichés, filler words	`rulesFor()`, `CLICHES`, `FILLERS` in `writing-check.js`
Records-request laws and response days	`LAWS` in `writing-reporting.js`
Revision passes, legal checklist	`PASSES`, `LEGAL` in `writing-revisions.js`
Release plan steps and their timing	`RELEASE_STEPS` in `music-business.js`
Song statuses, PROs, keys	`STATUSES`, `PROS`, `KEYS` in `music-songs.js`
Screenplay margins, element order, lines per page	`TYPES`, `TAB_ORDER`, `LINES_PER_PAGE` in `screenplay.js`
Revision colours, dual-dialogue columns, breakdown categories	`REV_COLORS`, `DUAL`, `BREAKDOWN` in `screenplay.js`
How often the sync folder is checked	`EVERY` in `sync.js`
Packaging platforms, character limits and title formulas	`PLATFORMS`, `TITLE_FORMULAS` in `packaging.js`
Deliverables per platform (and their Resolve settings)	`DELIVERABLES` in `packaging.js`
Marker colours for each kind of block	`MARKER_KINDS` in `timeline-export.js` (colour names from `MARKER_COLORS` in `media-core.js`)
Research statuses	`STATUSES` in `research.js`
Team starter roles	`STARTER_ROLES` in `team.js`
Paper edit structures	`STRUCTURES` in `media-core.js`
Storyboard frame shapes, shot types, camera moves	`ASPECTS`, `SHOT_TYPES`, `MOVES` in `storyboarder.js`
Zine sizes and presets	`SIZES`, `PRESETS` in `zine-planner.js`
Drawing colours and fonts	`COLORS`, `FONTS` in `sketch-core.js`
Default resources	Replace `default-resources-cobbler-project.csv` at the top of the repository (on GitHub: Add file › Upload files, drop it in, Commit changes). The online app reads it straight away, give or take GitHub's few-minute cache; the app ZIP rebuilds the double-click copy itself. Editing locally? Run `./update-resources.sh` too.
Fonts	Section 1 of `style.css` (Faculty Glyphic from Google Fonts)

9. Exports and file formats

- **Timelines** (`timeline-export.js`): FCPXML 1.9, Premiere XML (xmeml 4) and CMX3600 EDL, built from the paper edit, plus a markers-only EDL in DaVinci Resolve's format (`|C:ResolveColorBlue |M:name |D:frames`).
- **Reading from Resolve** (`media-core.js`): `parseMarkers()` reads Resolve's marker EDL or an Edit Index CSV; `parseTranscript()` reads its subtitle .srt (moving times that start at 01:00:00 back to 00:00) and frame-based timecodes.
- **Manuscripts** (`writing-draft.js`, `manuscriptDocx()`): a .docx in standard manuscript format, built directly (with a page-number header from page two).
- **Sheet music**: ABC notation drawn by abcjs; PDFs print the drawn notation with the lyrics (`music-songs.js`, `songPDF()`).
- **Credits** (`team-tasks.js`): drawn on a canvas and saved as PNG (cards zipped with `zip.js`).
- **R code** (`data-analysis.js`, `rCode()` and `datawrapperCode()`): written from fixed templates on the page, without the internet. The Datawrapper version uses the httr2 package and Datawrapper's API (create chart, upload CSV, publish), reading the token from `DATAWRAPPER_API_KEY`.
- **Calendars** (.ics; the shared calendar adds a calendar name and a refresh interval, and every event has a fixed UID so updates replace it), **contacts** (.vcf), **CSV** tables, **Markdown** pitches and scripts.
- **PDFs** come from print layouts through the print window ("Save as PDF"); **.docx** files are built by `doc-export.js`.

10. The guides and the app ZIP

These guides are HTML files in `resources/docs/` (`user-guide.html`, `code-guide.html`, styled by `docs.css`). After editing them, rebuild the PDFs (needs Google Chrome installed):

```
./make-docs.sh
```

Download the blank app (.zip) in Settings collects every file the pages use (found by following the sidebar's links and each page's scripts and styles) plus the README, change log, scripts and guides. It only works when the app is opened from a local server, because browsers don't let pages read files from the disk directly. It contains no projects: those live in your browser.

11. Tips

- Back up all projects (Projects button → Backup → All projects) before changing how data is saved.
- If a page stops working, open the developer tools' Console: the error names the file and line.
- Keep the style of the surrounding code: comments explaining what each part does, and plain, readable names.
- When a page's data shape changes, make older data still load (most scripts "fill in anything missing" right after reading their data; copy that pattern).

- Record what you changed in `change-log.md`.